

Apache OpenOffice

Version 4.1

Calc Guide

AOO Documentation Team

Chapter **13**

Calc as a Simple Database

*A guide for users and macro
programmers*

Copyright

This document is Copyright © 2025 by The Apache Software Foundation. You may distribute it and/or modify it under the terms of the Creative Commons Attribution License, version 3.0 or later (<https://creativecommons.org/licenses/by/3.0/>).

Apache, Apache OpenOffice, and OpenOffice.org are trademarks of the Apache Software Foundation. Used with permission.

Acknowledgments

This document is updated from a previous version by the OOoAuthors team. The AOO Documentation Team appreciates the many contributions of Michael Scarborough and Michele Petrovsky.

Publication date and software version

Published August 25, 2025. Based on Apache OpenOffice 4.1.

Note for Mac Users

Some keystrokes and menu items differ on a Mac from those used in Windows and Linux. The table below gives some common substitutions for the instructions in this chapter. For a more detailed list, see the application Help.

<i>Windows/Linux</i>	<i>Mac equivalent</i>	<i>Effect</i>
Tools > Options menu selection	OpenOffice > Preferences	Access setup options
<i>Right-click</i>	<i>Control+click</i>	Open context menu
<i>Ctrl</i> (Control)	⌘ (Command)	Used with other keys
<i>F5</i>	<i>Shift+⌘+F5</i>	Open the Navigator
<i>F11</i>	<i>⌘+T</i>	Open Styles & Formatting window

Contents

Chapter 13

Calc as a Simple Database	1
Introduction.....	4
Example Data.....	4
Associating a Range with a Name.....	5
Named Range.....	5
Database Range.....	6
Sorting.....	7
Filters.....	9
Auto Filters.....	9
Standard Filters.....	10
Advanced Filters.....	11
Manipulating Filtered Data.....	12
Calc Functions Similar to Database Functions.....	12
Count and Sum Cells that Match Conditions: COUNTIF, COUNTIFS, SUMIF, SUMIFS, and SUMPRODUCT.....	14
Ignore Filtered Cells Using SUBTOTAL.....	17
Using Formulas to Find Data.....	18
ADDRESS Returns a String with a Cell's Address.....	21
INDIRECT Converts a String to a Cell or Range.....	21
INDEX Returns Cells Inside a Specified Range.....	23
Database-specific Functions.....	24
Macros for a Calc Database.....	26
Macros for Named Ranges.....	26
Macro for a Database Range.....	28
Macros for Filtering.....	29

Introduction

A Calc document has several database-like capabilities, providing sufficient functionality to satisfy the needs of many users for a simple database, without having to master the more complex and much more powerful tools available in OpenOffice Base. This chapter presents the capabilities of a Calc document that make it suitable as a database tool. Where applicable, the functionality is explained using the GUI (Graphical User Interface) and macros. Because macros are a more advanced topic, the uses of the GUI and standard Calc functions are explained first; macros are reserved for the end of the chapter.

In a database, a record is a group of related data items treated as a single unit of information. Each item in the record is called a field. A table consists of records. Each record in a table has the same structure. A table can be visualized as a series of rows and columns. Each row in the table corresponds to a single record, and each column corresponds to the fields. A spreadsheet in a Calc document is similar in structure to a database table. Each cell corresponds to a single field in a database record. For many people, Calc implements sufficient database functionality so that no other database program or functionality is required.

Example Data

Many of the examples in this chapter refer to the data in Table 1. If you are reading this chapter as a Writer document, you can copy the data from Table 1 directly to a spreadsheet. Select the cells containing the data, excluding the headers A-G and the row numbers 1-16, then copy and paste them into a Calc document.

A spreadsheet might be used as a grading program while teaching. Each row represents a single student, and the columns represent the grades received on homework, labs, and tests. The strong calculation capability provided in a spreadsheet makes this an excellent choice.

Table 1. Simple grading spreadsheet

	A	B	C	D	E	F	G
1	Name	Test 1	Test 2	Quiz 1	Quiz 2	Average	Grade
2	Andy	95	93	93	92	93.25	
3	Betty	87	92	65	73	79.25	
4	Bob	95	93	93	92	93.25	
5	Brandy	45	65	92	85	71.75	
6	Frank	95	93	85	92	91.25	
7	Fred	87	92	65	73	79.25	
8	Ilsab	70	85	97	79	82.75	
9	James	45	65	97	85	73	
10	Lisa	100	97	100	93	97.5	
11	Michelle	100	97	100	65	90.5	
12	Ravi	87	92	86	93	89.5	
13	Sal	45	65	100	92	75.5	

	A	B	C	D	E	F	G
14	Ted	100	97	100	85	95.5	
15	Tom	70	85	93	65	78.25	
16	Whil	70	85	93	97	86.25	

Tip

Although the choice to associate a row to a record rather than a column is arbitrary, it is almost universal. In other words, you are not likely to hear someone refer to a column of data as a single database record.

Associating a Range with a Name

In a Calc document, a range refers to a contiguous, that is, a side-by-side group of cells. A range must contain at least one cell. You can associate a meaningful name to a range, which allows you to refer to the range using the meaningful name. You can create either a *database range* with some database-like functionality or a *named range* that does not. A name is usually associated with a range for one of three reasons:

- 1) Associating a range with a name enhances readability by using a meaningful name.
- 2) If a range is referenced by name in multiple locations, you can point the name to another location, and all references point to the new location.
- 3) Ranges associated to a name are shown in the Navigator, accessible by pressing the *F5* key or clicking on the  icon in either the main toolbar or the Sidebar. The Navigator allows for quick navigation to the associated ranges.

Named Range

The most common usage of a named range is, as that wording implies, to associate a range of cells with a meaningful name. For example, create a range named *Scores*, and then use the following equation: `=SUM(Scores)`. To create a named range, select the range to define. Typically, you would not include a column or row label with this kind of named range, so the SUM function only gets numeric values. Use **Insert > Names > Define** to open the Define Names dialog (see Figure 1). Use the Define Names dialog to add and modify one named range at a time.

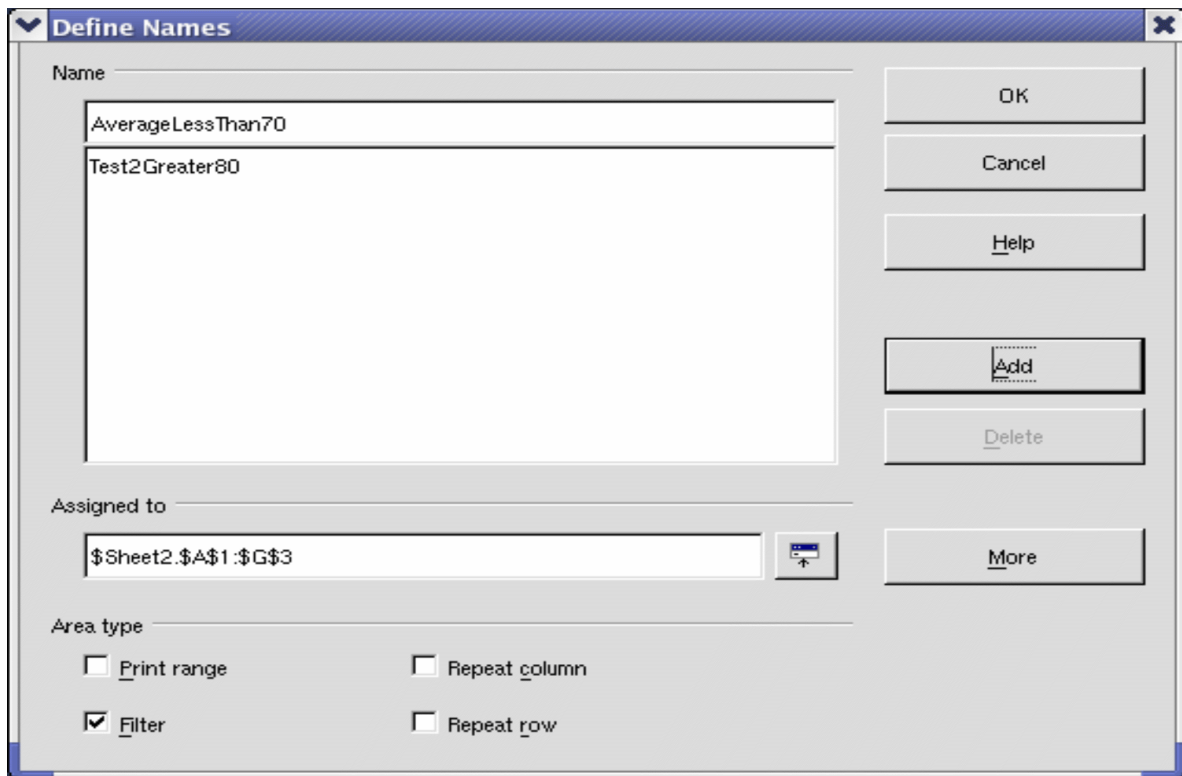


Figure 1. Define a named range.

A set of named ranges can be defined in one step, using the text in the column or row headers as the range names. Select the range including the headers and the data, then use **Insert > Names > Create** to open the Create Names dialog (see Figure 2), which allows you to simultaneously create multiple named ranges based on the top row, bottom row, right column, or left column. If, for example, you choose to create ranges based on the top row, one named range is created for each column header—the header is not included in the named range. Although the header is not included in the range, the text in the header is used to name the range.

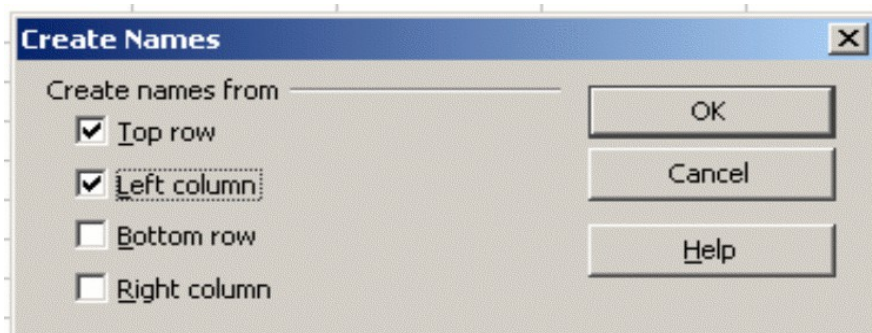


Figure 2. Define a named range.

Database Range

A database range provides tools that are useful when performing database-related activities. For example, you can mark the first row as headings. The range will also

store the settings of the last sort and filter actions so that these can be repeated without reentering the settings. To create, modify, or delete a database range, use **Data > Define Range** to open the Define Data Range dialog (see Figure 3). When you first define a range, the Modify button in the example is labeled Add.

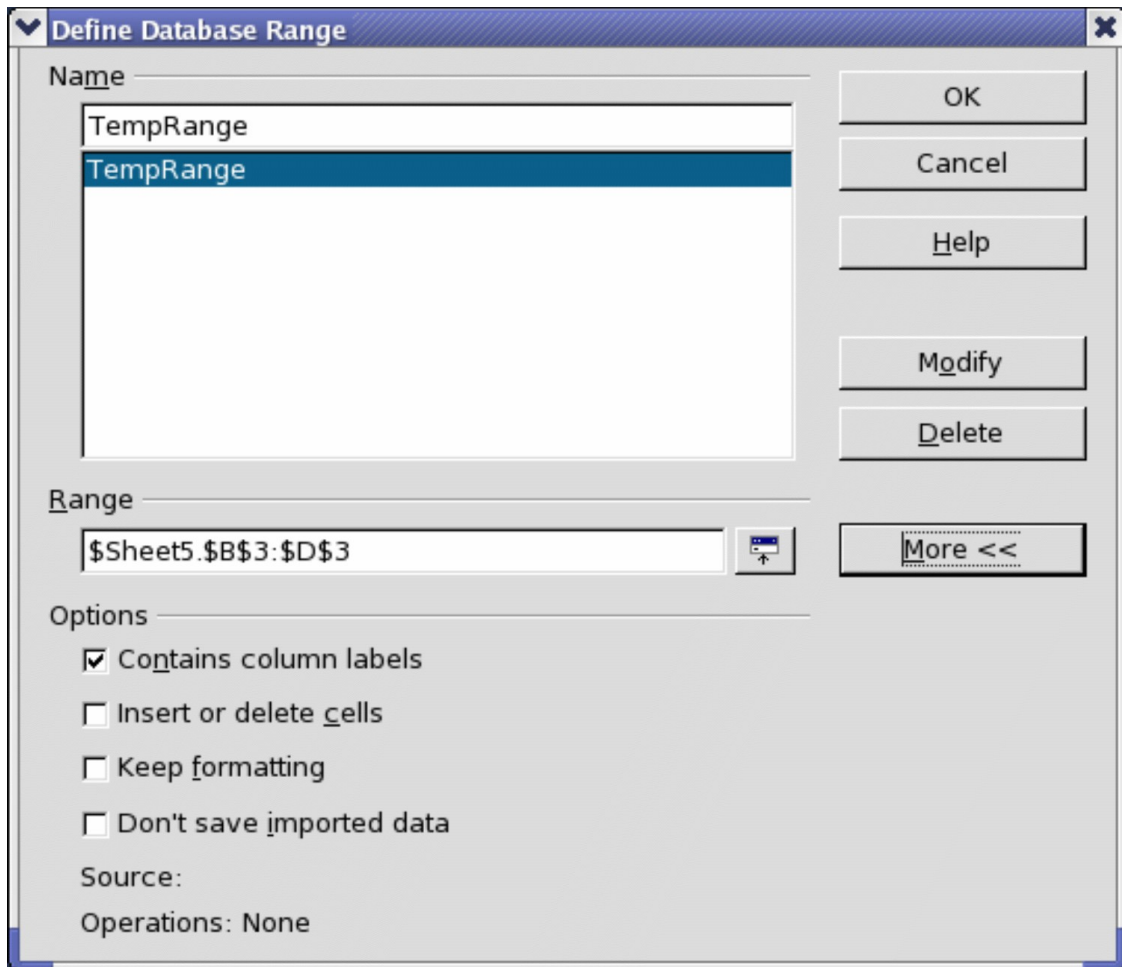


Figure 3. Define a database range.

Sorting

The sorting mechanism in a Calc document rearranges the data in the sheet. The first step in sorting data is selecting the data you want to sort. To sort the data in Table 1, select the cells from A1 to G16—if you include the column headers, indicate this in the sort dialog (see Figure 5). Use **Data > Sort** to open the Sort dialog (see Figure 4). You can sort by up to three columns or rows at a time.

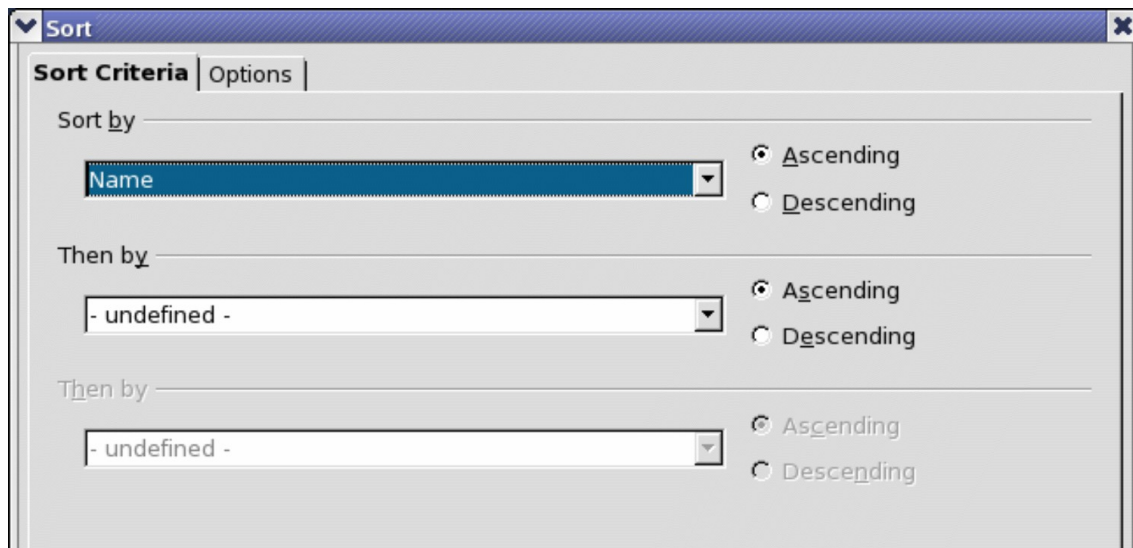


Figure 4. Sort by the Name column.

Click on the Options tab (see Figure 5) to set the sort options. Check the **Range contains column labels** checkbox to prevent column headers from being sorted with the rest of the data. The Sort by list box in Figure 4 displays the columns using the column headers if the **Range contains column labels** checkbox in Figure 5 is checked. If the **Range contains column labels** checkbox is not checked however, then the columns are identified by their column name, Column A, for example.

Normally, sorting data replaces existing data with the newly sorted data. The **Copy sort results to** checkbox, however, leaves the selected data unchanged, and a copy of the sorted data is copied to the specified location. You can directly enter a target address (Sheet3.A1, for example) or select a predefined range.

Check the **Custom sort order** checkbox to sort based on a predefined list of values. To set predefined lists, use **Tools > Options > OpenOffice Calc > Sort Lists** and enter your sort lists. Predefined sort lists are useful for sorting lists of data that should not be sorted alphabetically or numerically. For example, sorting days based on their name.

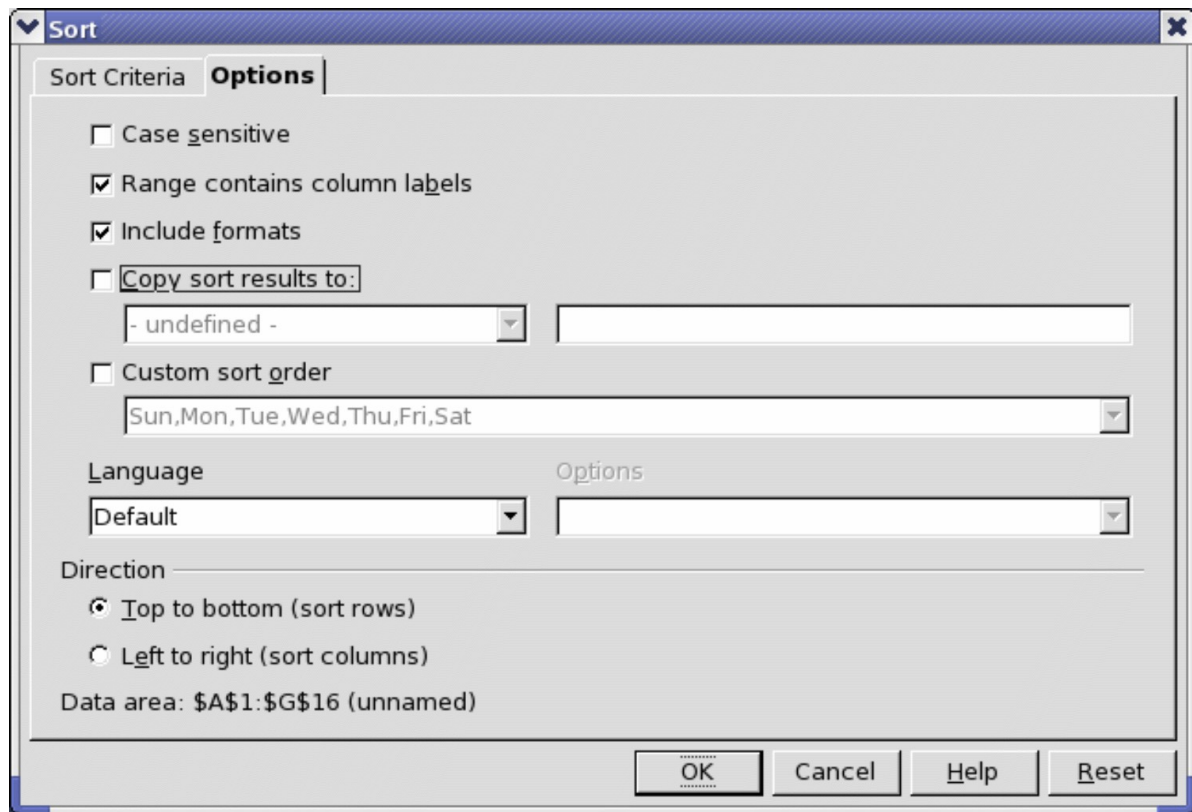


Figure 5. Set sort options.

Caution



When a cell is moved during a sort operation, external references to that cell are not updated. If a cell that contains a relative reference to another cell is moved, the reference is relative to the new position when sorting is finished. Know the behavior of references during sorting, and do not be alarmed; this is almost always what you want because the reference is to the right or left in the same row.

Filters

Use filters to limit the visible rows in a spreadsheet. Generic filters, common to all sorts of data manipulations, are automatically provided by the auto filter capability. You can also define custom filters.

Auto Filters

Use auto filters to quickly create easily accessible filters like those commonly used in many different types of applications. After creating an auto filter for a specific column, a combo box is added to the column. The combo box provides quick access to each of the auto filter types.

- The All auto filter causes all rows to be visible.
- The Standard auto filter opens the Standard Filter dialog, which is the same as the standard filter.

- The Top 10 auto filter displays the ten rows with the largest value. If the value 70 is in the top ten values, then all rows containing the value 70 in the filtered column are displayed. In other words, more than ten rows may be displayed.
- An auto filter entry is created for each unique entry in the column.

To create an auto filter, first select the columns to filter. For example, using the data in Table 1, select data in column D. If you do not select the title rows, Calc asks if the title row or the current row should be used. Although you can place the auto filter in any row, only the rows below the auto filter are filtered. Use **Data > Filter > AutoFilter** to insert the auto filter combo box in the appropriate cell. Finally, use the drop-down arrow to choose an appropriate auto filter (see Figure 6).

You can add an AutoFilter to every column in a cell range by clicking on a single cell in the range and selecting **Data > Filter > AutoFilter**. Calc will automatically detect the surrounding area of contiguously filled cells and add an AutoFilter to every column.

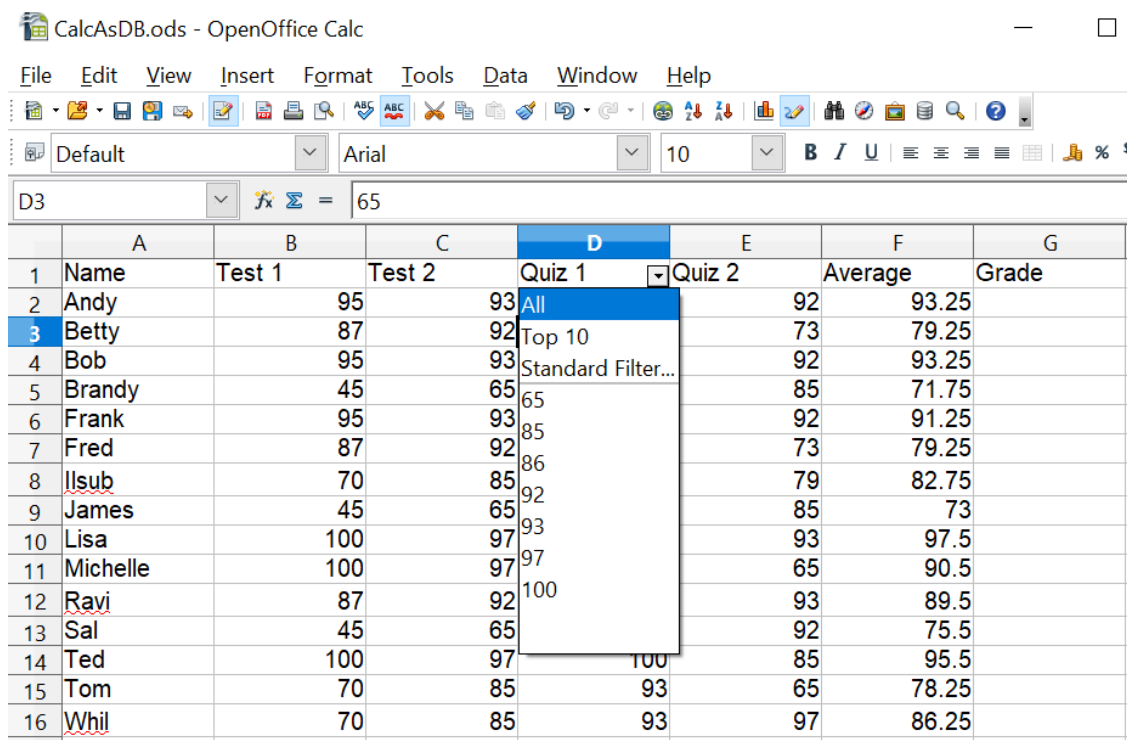


Figure 6: Use an auto filter with column C

Remove an auto filter by repeating the steps to create it—in other words, the menu option acts as a toggle to turn the auto filter on and off. When an auto filter is removed, the combo box is removed from the cell. The menu **Data > Filter** also has an option to *Hide AutoFilter*, which also removes the filter's drop-down list.

Standard Filters

Use **Data > Filter > Standard Filter** to open the Standard Filter dialog (see Figure 7) and limit the view based on one or more filter conditions. Use **Data > Filter > Remove Filter** to turn off the filter.

You can set multiple conditions referring to one or more columns. The conditions can be linked by either an OR or AND condition. It is important to set the link correctly. In Figure 7, the filter has been set to allow two values for the *Quiz 1* column. If these

two conditions were linked by AND, the filter would hide every row because a cell cannot have values of 67 and 77 simultaneously.

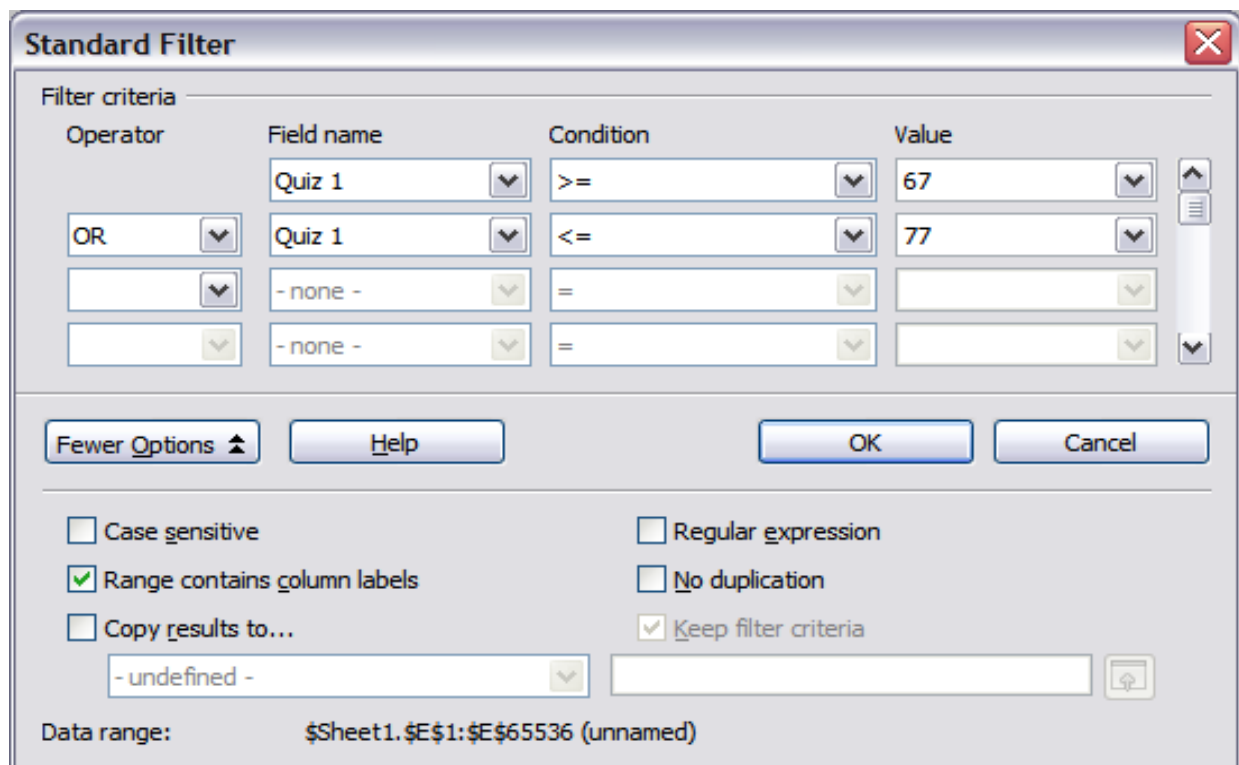


Figure 7: Use the standard filter

Clicking the **More Options** button in the Standard Filter dialog displays several useful options. Selecting *Copy results to...* allows you to send the filter results to another location. The *Regular expression* option allows using wildcard text matching. The *No duplication* option will display only unique rows if the filter results are sent to a different location.

Advanced Filters

An advanced filter supports up to eight filter conditions. The criteria for an advanced filter is stored in a sheet. The first step in creating an advanced filter is entering the filter criteria into the spreadsheet.

- 1) Select an empty space in the Calc document. The empty space may reside in any sheet in any location in the Calc document.
- 2) Duplicate the column headings from the area to be filtered into the area that will contain the filter criteria.
- 3) Enter the filter criteria underneath the column headings (see Table 2). The criterion in each column of a row is connected with AND. The criteria from each row are connected with OR.

Table 2. Example advanced filter criteria

Name	Test 1	Test 2	Quiz 1	Quiz 2	Average	Grade
= "Andy"		>80				
					<80	

Tip

Define named ranges to reference your advanced filter criteria and any destination ranges for filtered data (see Figure 1). Each appropriately configured named range is available in drop-down list boxes in the Advanced Filter dialog (see Figure 8).

After creating the filter criteria, apply an advanced filter as follows:

- 1) Select the sheet ranges that contain the data to be filtered.
- 2) Use **Data > Filter > Advanced Filter** to open the Advanced Filter dialog (see Figure 8).
- 3) Select the range containing the filter criteria and any other relevant options.
- 4) Click **OK**.

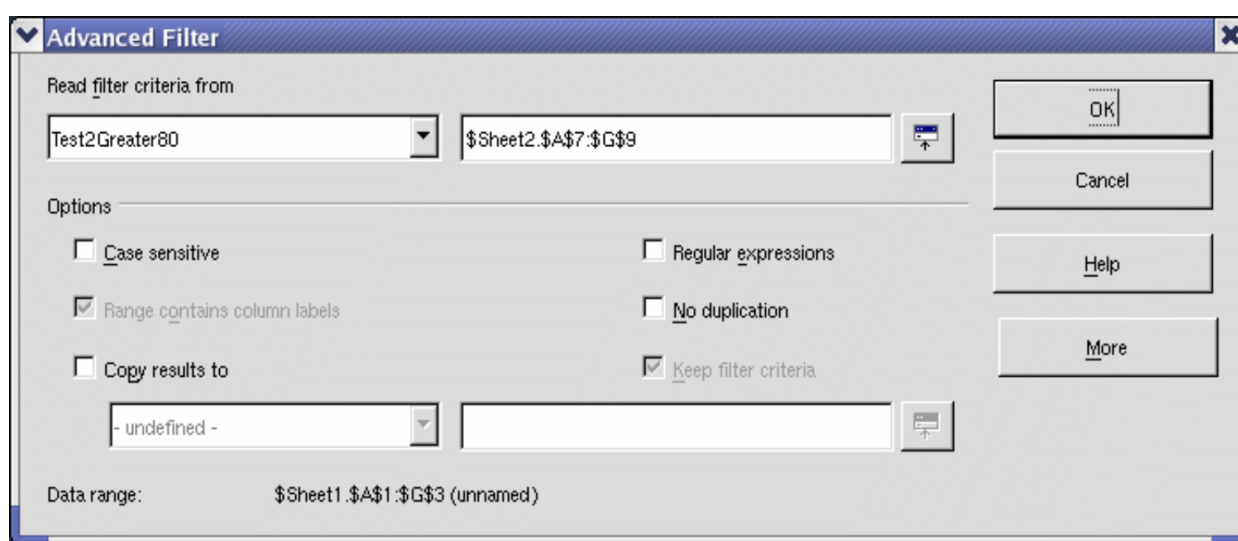


Figure 8. Apply an advanced filter using a previously defined named range.

Manipulating Filtered Data

Filtered data copied to a new location may be selected, modified, and deleted at will. However, data that is not copied requires special attention because rows that do not match the filter criteria are hidden. OpenOffice behaves differently depending on how the cells were hidden and the operation done.

Cells may be hidden using an outline, data filter, or the hide command. When data is moved by dragging or using cut and paste, all of the cells are moved, including the hidden cells. When copying data, however, filtered data includes only the visible cells, and data hidden using an outline or the hide command copies all of the data.

Calc Functions Similar to Database Functions

Although every Calc function can be used for database manipulation, the functions in Table 3 are those most frequently used in that way. Some functions' names differ only by the letter appended at the end; AVERAGE and AVERAGEA, for example. Functions that do not end with the letter A operate only on numeric values, and cells that

contain text or are empty are ignored. The corresponding function, whose name ends with the letter A, treats text values as a number with the value of zero; blank cells are still ignored.

Table 3. Functions frequently used as database functions.

Function	Description
AVERAGE	Return the average. Ignore empty cells and cells that contain text.
AVERAGEA	Return the average. The value of text is 0 and empty cells are ignored.
COUNT	Count the number of numeric entries; text entries are ignored.
COUNTA	Count the number of non-empty entries.
COUNTBLANK	Return the number of empty cells.
COUNTIF COUNTIFS	Return the number of cells that meet the search criteria. COUNTIF accepts one criterion, COUNTIFS accepts multiple criteria.
HLOOKUP	Search for a specific value across the columns in the first row of an array. Return the value from a different row in the same column.
INDEX	Return the content of a cell, specified by row and column number or an optional range name.
INDIRECT	Return the reference specified by a text string, e.g., "C26".
LOOKUP	Return the contents of a cell either from a one-row or one-column range or from an array.
MATCH	Search an array and return the relative position of the found item.
MAX	Return the maximum numeric value in a list of arguments.
MAXA	Return the maximum numeric value in a list of arguments. The value of text is 0.
MIN	Return the minimum numeric value in a list of arguments.
MINA	Return the minimum numeric value in a list of arguments. The value of text is 0.
MEDIAN	Return the median of a set of numbers.
MODE	Return the most common value in a data set. If several values have the same frequency, it returns the smallest value. An error occurs when all values are unique.
OFFSET	Return a cell range or the value of a single cell offset by a certain number of rows and columns from a given reference point and spanning a given number of rows and columns. In other words, locates a cell or the data in it that lies x rows and y columns away from the starting point you give it.
PRODUCT	Return the product of the cells.

Function	Description
STDEV	Estimate the standard deviation based on a sample.
STDEVA	Estimate the standard deviation based on a sample. The value of text is 0.
STDEVP	Calculate the standard deviation based on the entire population.
STDEVPA	Calculate the standard deviation based on the entire population. The value of text is 0.
SUBTOTAL	Calculate a specified function based on a subset created using AutoFilters.
SUM	Return the sum of the cells.
SUMIF SUMIFS	Calculate the sum for the cells that meet the search criteria. SUMIF takes one criterion, SUMIFS accepts multiple criteria.
SUMPRODUCT	Can be used to calculate counts or sums that meet multiple criteria. The calculation does element-wise multiplication of the given cell ranges and then sums those products. See below for more details.
VAR	Estimate the variance based on a sample.
VARA	Estimate the variance based on a sample. The value of text is 0.
VARP	Calculate the variance based on the entire population.
VARPA	Calculate the variance based on the entire population. The value of a text is 0.
VLOOKUP	Search for a specific value across the rows in the first column of an array. Returns the value from a different column in the same row.

Most of the functions in Table 3 require no explanation, either because they are well understood (SUM, for example) or because if you need to use them, you know what they are (STDEV, for example). Unfortunately, some of the more useful functions are infrequently used because they are not well understood. For instance, stdev or standard deviation measures how widely a value differs from the average value or mean of the category. Variance and standard deviation are measures of how spread out data points are from the mean, but differ in how they are calculated. Variance is the average of the squared differences from the mean, while standard deviation is the square root of the variance. Essentially, standard deviation is the square root of variance.

Count and Sum Cells that Match Conditions: COUNTIF, COUNTIFS, SUMIF, SUMIFS, and SUMPRODUCT

The COUNTIF, COUNTIFS, SUMIF, and SUMIFS functions calculate their values based on search criteria. The search criteria can be a number, expression, text string, or even a regular expression. The search criteria can be contained in a referenced cell or included directly in the function call.

The COUNTIF function counts the number of cells in a range that match specified criteria. The first argument to COUNTIF specifies the range to search, and the second argument is the search criteria. Table 4 illustrates different search criteria using the COUNTIF function referencing the data shown in Table 1.

Tip

If you want to try the formulas using the data in Table 1 and you are reading this document in Writer, you can copy the data from Table 1 directly to a spreadsheet. Select the cells containing the data, excluding the headers A-G and the row numbers 1-16, then copy and paste them into a Calc document.

The COUNTIFS function works in much the same way, but it accepts multiple criteria. The function syntax accepts the first search range and then the first search condition. This is optionally followed by a second search range and a second condition. Additional ranges and conditions can also be added.

The first two arguments for SUMIF serve the same purpose as the arguments for COUNTIF; the range that contains the cells to search and the search criteria. The third and final argument for SUMIF specifies the range to sum. For each cell in the search range that matches the search criteria, the corresponding cell in the sum range is added to the sum.

To accommodate multiple criteria, the SUMIFS function has the range to sum as the first argument. This is followed by pairs of arguments listing the search ranges and the search conditions. For example,

```
=SUMIFS(C1:C10;A1:A10; ">2";B1:B10;"<4")
```

calculates the sum of cells in C1:C10 where the corresponding cells in A1:A10 are greater than 2 and the corresponding cells in B1:B10 are less than 4.

Table 4. Examples of search criteria for the COUNTIF and SUMIF functions.

Criteria Type	Function	Result	Description
Number	=COUNTIF(B1:C16; 95)	3	Finds numeric values of 95.
Text	=COUNTIF(B1:C16; "95")	3	Finds numeric or text values of 95.
Expression	=COUNTIF(B1:C16; ">95")	6	Finds numeric values greater than 95.
Expression	=COUNTIF(B1:C16; 2*45+5)	3	Finds only numeric values of 95.
Regular expression	=COUNTIF(B1:C16; "9.*")	12	Finds numbers or text that start with 9.
Reference a cell	=COUNTIF(B1:C16; B3)	3	Finds a number or a number and text depending on the data type in cell B3.
Regular expression	=SUMIF(A1:A16; "B.*"; B1:B16)	227	Sum Column B for names in Col. A starting with the letter B.

SUMPRODUCT can return either a sum or a count, and uses a different syntax from the COUNTIF and SUMIF functions. The basic function of SUMPRODUCT is to do element-wise multiplication of cell ranges and then add those products. For example, the formula =SUMPRODUCT(A1:A3; B1:B3) returns the result of $A1*B1 + A2*B2 + A3*B3$. That is, the first elements of all the ranges are multiplied, and the second elements of each range are multiplied, continuing to the end of the ranges. Then, the multiplication results are added. The example only has two ranges, but there could be more.

SUMPRODUCT's use for counting and summing relies on the fact that comparisons in Calc return numeric values. If a comparison is TRUE, the value returned is 1. If the comparison is FALSE, the value returned is 0. This is easily seen with a simple formula. If cell B1 contains the formula = A1 < 2, the value will be displayed as either TRUE or FALSE, depending on the value of A1. If you use the menu **Format > Cells** to format B1 to display an integer, the TRUE and FALSE conditions will be 1 and 0. With SUMPRODUCT, if you write =SUMPRODUCT(A1:A10 > 3), the comparison will return a 1 or a 0 for each position in A1:A10, depending on whether the cell is greater than 3. The ones and zeros are then summed (no multiplication occurs because there is only one range), resulting in a count of how many cells in A1:A10 are greater than 3.

If you use two ranges, as in =SUMPRODUCT(A1:A10 > 3; B1:B10 < 10), the comparison translates the values in the cells into ones and zeros, and then the element-wise multiplication occurs. If either condition is FALSE, the multiplication results in a zero. Only if both (all) conditions are TRUE does the multiplication result in a 1. When the multiplication results are added, we get a count of the rows where all the conditions were true.

To use SUMPRODUCT to compute a conditional sum, all that is required is that the column to be summed without any comparison. Its values then remain as they are,

not translated into ones and zeros. Table 5 shows the steps taken in calculating `=SUMPRODUCT(A1:A5 > 3; B1:B5 < 10; C1:C5)`. In a spreadsheet, the intermediate steps happen internally and are not displayed. The spreadsheet would have only the raw data and one cell containing the SUMPRODUCT formula. The intermediate steps are shown only to illustrate the logic of the calculation.

Table 5: Steps for Calculating SUMPRODUCT(A1:A5 > 3; B1:B5 < 10; C1:C5)

Raw Data			After Comparison				
A	B	C	A'	B'	C'	A' * B' * C'	Sum
5	9	7	1	1	7	7	13
3	8	5	0	1	5	0	
7	12	9	1	0	9	0	
2	18	4	0	0	4	0	
8	5	6	1	1	6	6	

An important advantage of using SUMPRODUCT to conditionally count or sum cells is that you can use any function within it that returns a TRUE or FALSE. For example, you might want to sum all the cells in B2:B100 where column A is blank. This is not easily done with SUMIF. With SUMPRODUCT, you can use the function ISBLANK and easily get the result: `=SUMPRODUCT(ISBLANK(A2:A100); B2:B100)`. There are several IS* functions, such as ISERROR, ISNUMBER, ISTEXT (see the Calc Help for a complete list), that can be combined with SUMPRODUCT in powerful ways.

Ignore Filtered Cells Using SUBTOTAL

The SUBTOTAL function applies a function (see Table 6) to a range of data, but it ignores cells hidden by a filter and cells that already contain a SUBTOTAL. For example, `=SUBTOTAL(2, "B2:B16")` counts the number of cells in B2:B16 that are not hidden by a filter.

Table 6. Function index for the SUBTOTAL function.

Function index	Function
1	AVERAGE
2	COUNT
3	COUNTA
4	MAX
5	MIN
6	PRODUCT
7	STDEV
8	STDEVP
9	SUM
10	VAR
11	VARP

Tip

Do not forget that the SUBTOTAL function ignores cells that use the SUBTOTAL function. Let's say you have a spreadsheet that tracks investments. Retirement investments are grouped together with a subtotal. The same is true of regular investments. You can use a single subtotal that includes the entire range without worrying about the subtotal cells.

Using Formulas to Find Data

Calc offers numerous methods to find data in a sheet. For example, **Edit > Find & Replace** moves the display cursor based on simple and advanced searching. Use **Data > Filter** to limit what is displayed rather than simply moving the cursor. Calc also offers lookup functions used in formulas, for example, a formula to look up a student's grade based on their test score.

Search a Block of Data Using VLOOKUP

Use VLOOKUP to search the first column (columns are vertical) of a block of data and return the value from another column in the same row. For example, search the first column for the name "Fred" and then return the value in the cell two columns to the right. VLOOKUP supports two forms:

```
VLOOKUP(search_value; search_range; return_column_index)
VLOOKUP(search_value; search_range; return_column_index;
sort_order)
```

The first argument, `search_value`, identifies the value to find. The search value can be text, a number, or a regular expression. For example, **Fred** searches for the text Fred, **4** searches for the number 4, and **F.*** is the regular expression for finding something that starts with the letter F.

The second argument, `search_range`, identifies the cells to search; only the first column is searched. For example, **B3:G10** searches the same sheet containing the VLOOKUP formula, and **Sheet2.B3:G10** searches the range B3:G10 on Sheet2.

The `return_column_index` identifies the column to return; a value of 1 returns the first column in the range. The statement `=VLOOKUP("Bob"; A1:G9; 1)` finds the first row in A1:G9 containing the text **Bob**, and returns the value in the first column. The first column is the searched column, so the text **Bob** is returned. If the column index is 2, then the value in the cell to the right of Bob is returned: column B.

The final column, `sort_order`, is optional. The default value for `sort_order` is 1, which specifies that the first column is sorted in ascending order; a value of 0 specifies that the data is not sorted. A non-sorted list is searched by sequentially checking every cell in the first column for an exact match. If an exact match is not found, the text **#N/A** is returned.

A more efficient search routine is used if the data is sorted in ascending order. If one exact match exists, the returned value is the same as for a non-sorted list, but it is faster. If a match does not exist, the largest value in the column that is less than or equal to the search value is returned. For example, searching for 7 in (3, 5, 10) returns 5 because 7 is between 5 and 10. Searching for 27 returns 10, and searching for 2 returns **#N/A** because there is no match and no value less than 2.

If `sort_order` is omitted or set to 1 and the first column is not sorted in ascending order, the returned value is unpredictable. This is a common mistake; it may be hard to detect because an error is not identified, and the returned value may be correct

when the formula is first used. It is a good idea to set `sort_order` explicitly in all cases.

Use VLOOKUP when:

- The data is arranged in rows, and you want to return data from the same row, for example, student names with test and quiz scores to the right of the student's name.
- Searching the first column of a range of data.

Search a Block of Data Using HLOOKUP

Use HLOOKUP to search the first row (rows are horizontal) of a block of data and return the value from a row in the same column. HLOOKUP supports the same form and arguments as VLOOKUP:

```
HLOOKUP(search_value; search_range; return_row_index)
HLOOKUP(search_value; search_range; return_row_index;
sort_order)
```

Use HLOOKUP when:

- The data is arranged in columns, and you want to return data from the same column, for example, student names with test and quiz scores underneath the student's name.
- Searching the first row of a range of data.

Search a Row or Column Using LOOKUP

LOOKUP is similar to HLOOKUP and VLOOKUP. The search range for the LOOKUP function is a single *sorted* row or column. LOOKUP has two forms:

```
LOOKUP(search_value; search_range)
LOOKUP(search_value; search_range; return_range)
```

The search value is the same as HLOOKUP and VLOOKUP. The search range, however, must be a single row or a single column; for example, A7:A12 (values in column A) or C5:Q5 (values in row 5). If the return_range is omitted, the matched value is returned. Using LOOKUP without a return range is the same as using HLOOKUP or VLOOKUP with a column index of 1, except that LOOKUP always assumes the data is sorted. Using LOOKUP on a search range that is not sorted can produce incorrect results.

The return range must be a single row or column containing the same number of elements as the search range. If the search value is found in the fourth cell in the search range, then the value in the fourth cell in the return range is returned. The return range can have a different orientation than the search range. In other words, the search range can be a row, and the return range may be a column.

Use LOOKUP when:

- The search data is sorted in ascending order.
- The search data is not stored in the same row, column, or orientation as the return data.

Use MATCH to Find the Index of a Value in a Range

Use MATCH to search a single row or column and return the position that matches the search value. Use MATCH to find the index of a value in a range. The supported forms for MATCH are as follows:

```
=MATCH(search_value; search_range)
=MATCH(search_value; search_range; search_type)
```

The search value and search range are the same as for LOOKUP. The final argument, search type, controls how the search is performed. A search type of 1, sorted in ascending order, is the default. A search type of -1 indicates that the list is sorted in descending order. A search type of 0 indicates that the list is not sorted. Regular expressions can only be used on an unsorted list.

Use MATCH when:

- You need an index into the range rather than the value. By index, we mean the position within a set of values that a specific value occupies.
- The search data is in descending order, and the data is large enough that it must be searched assuming that it is sorted, because it is faster to sort a sorted list.

Examples

Consider the data in Table 1. Each student's information is stored in a single row. Write a formula to return Fred's average grade. The problem can be restated as: search column A in the range A1:G16 for Fred and return the value in column F (column F is the sixth column). The obvious solution is

```
=VLOOKUP("Fred"; A2:G16; 6). Just as useful is
```

```
=LOOKUP("Fred"; A2:A16; F2:F16).
```

It is common for the first row in a range to contain column headers. All search functions check the first row to see if there is a match and then ignore it if it does not, in case the first row is a header.

What if the column heading **Average** is known, but not the column containing the average? Find the column containing Average rather than hard-coding the value 6. A slight modification using MATCH to find the column yields

```
=VLOOKUP("Fred"; A2:G16; MATCH("Average"; A1:G1; 0)); notice that the heading is not sorted. As an exercise, use HLOOKUP to find Average and then MATCH to find the row containing Fred.
```

As a final example, write a formula to assign grades based on a student's average score. Assume that a score less than 51 is an F, less than 61 is an E, less than 71 is a D, less than 81 is a C, less than 91 is a B, and 91 to 100 is an A. Assume that the values in Table 7 are in Sheet2.

Table 7. Associate scores to a grade.

	A	B
1	Score	Grade
2	0	F
3	51	E
4	61	D
5	71	C
6	81	B
7	91	A

The formula `=VLOOKUP(83; $Sheet2.$A$2:$B$7; 2)` is a solution.

This will return B because 81 is the last value less than or equal to 83, and B is in the second column of Sheet2.A2:B7. Dollar signs are used so that the formula can be copied and pasted to a different location and still reference the same values in Table 7.

ADDRESS Returns a String with a Cell's Address

Use ADDRESS to return a text representation of a cell address based on the row, column, and sheet; ADDRESS is frequently used with MATCH. The supported forms for ADDRESS are as follows:

```
ADDRESS(row; column)
ADDRESS(row; column; abs)
ADDRESS(row; column; abs; sheet)
```

The row and column are integer values where ADDRESS(1; 1) returns **\$A\$1**. The abs argument specifies which portion is considered absolute and which portion is considered relative (see Table 8); an absolute address is specified using the \$ character. The sheet is included as part of the address only if the sheet argument is used. The sheet argument is treated as a string. Using ADDRESS(MATCH("Bob";A1:A5 ; 0); 2) with the data in Table 1 returns **\$B\$4** because the row is set by MATCH("Bob";A1:A5 ; 0) and it returns 4, and the column is set to 2.

Tip

Calc supports numerous powerful functions that are not discussed here. For example, the ROW, COLUMN, ROWS, and COLUMNS statements are not discussed.

Table 8. Values supported by the abs argument to ADDRESS.

Value	Description
1	Use absolute addressing. This is the default value if the argument is missing or an invalid value is used. ADDRESS(2; 5; 1) returns \$E\$2.
2	Use an absolute row reference and a relative column reference. ADDRESS(2; 5; 2; "Blah") returns Blah.E\$2.
3	Use a relative row reference and an absolute column reference. ADDRESS(2; 5; 3) returns \$E2.
4	Use relative addressing. ADDRESS(2; 5; 4) returns E2.

INDIRECT Converts a String to a Cell or Range

Use INDIRECT to convert a string representation of a cell or range address to a reference to the cell or range. Table 9 contains examples accessing data as shown in Table 7.

Table 9. Examples using INDIRECT.

Example	Comment
INDIRECT("A2")	Returns cell A2, which contains 0 .
INDIRECT(G1)	If Cell G1 contains the text A2, then this returns 0 .
SUM(INDIRECT("A2:A7"))	Returns the sum of the range A2:A7, which is 355 .
INDIRECT(ADDRESS(2; 1))	Returns the contents of cell \$A\$2, which is 0 .

OFFSET Returns a Cell or Range Offset From Another

Use OFFSET to return a cell or range offset by a specified number of rows and columns from a given reference point. The first argument specifies the reference point. The second and third arguments specify the number of rows and columns to move from the reference point, in other words, where the new range starts. The OFFSET function has the following syntax:

```
OFFSET(reference; rows; columns)
OFFSET(reference; rows; columns; height)
OFFSET(reference; rows; columns; height; width)
```

Tip

If the width or height is greater than one, the OFFSET function returns a range. If both the width and height are missing or equal 1, a cell value is returned.

If the height or width is missing, they default to 1. Using values from Table 1, Listing 1 uses OFFSET to obtain the sum of the quiz scores for the student named Bob.

Listing 1. Complex example of OFFSET.

```
=SUM(OFFSET(INDIRECT(ADDRESS(MATCH("Bob";A1:A16; 0); 4)); 0; 0; 1; 2))
```

In its entirety, Listing 1 is complex and difficult to understand. Table 10 isolates each function in Listing 1, making the listing more understandable.

Table 10. Breakdown of Listing 1.

Function	Description
MATCH("Bob";A1:A16; 0)	Return 4 because Bob is the fourth entry in column A.
ADDRESS(4; 4)	Return "\$D\$4".
INDIRECT("\$D\$4")	Convert "\$D\$4" into a reference to the cell D4.
OFFSET(\$D\$4; 0; 0; 1; 2)	Return the range D4:E4.
SUM(D4:E4)	Return the sum of Bob's quiz scores.

Although Listing 1 works as intended, it breaks easily and unexpectedly. Consider, for example, what happens if the range is changed to A2:A16. MATCH returns an offset

into the provided range, so `MATCH("Bob";A2:A16 ; 0)` returns 3 rather than 4. `ADDRESS(3; 4)` returns `$D$3` rather than `$D$4`, and Betty's quiz scores are returned instead of Bob's. Listing 2 uses a slightly different method to obtain Bob's quiz scores.

Listing 2. Better use of OFFSET.

```
=SUM(OFFSET(A1; MATCH("Bob"; A1:A16; 0)-1; 3; 1; 2))
```

Table 11 contains a description of each function used in Listing 2. To see that Listing 2 is better than Listing 1, replace A1 with A2 in Listing 2, and notice that you still obtain Bob's quiz scores.

Table 11. Breakdown of Listing 2.

Function	Description
<code>MATCH("Bob";A1:A16; 0)-1</code>	Return 3 because Bob is the fourth entry in column A.
<code>OFFSET(A1; 3; 3; 1; 2)</code>	Return the range D4:E4.
<code>SUM(D4:E4)</code>	Return the sum of Bob's quiz scores.

Tip

The first argument to `OFFSET` can be a range, so you can use a defined range name.

Using formulas as complex as the `OFFSET` examples with the name Bob coded in the formula is not a good solution, and was used here only for clarity. It would be much more efficient to write the text Bob in a cell, say H2, and write the `MATCH` part of the function as `MATCH($H$2; A1:A16; 0)`. The text in H2 could then be conveniently changed to bring up any student's quiz score.

INDEX Returns Cells Inside a Specified Range

`INDEX` returns the cells specified by a row and column number. The row and column number are relative to the upper left corner of the specified reference range. For example, using `=INDEX(B2:D3; 1; 1)` returns the cell B2. Table 12 shows the syntax for using the `INDEX` function.

Table 12. Syntax for INDEX.

Syntax	Description
<code>INDEX(reference)</code>	Return the entire range.
<code>INDEX(reference; row)</code>	Return the specified row in the range.
<code>INDEX(reference; row; column)</code>	Return the cell specified by row and column. A row and column of 1 returns the cell in the upper left corner of the range.
<code>INDEX(reference; row; column; range)</code>	A reference range can contain multiple ranges. The range argument specifies which range to use.

The `INDEX` function can return an entire range, a row, or a single column (see Table 12). The ability to index based on the start of the reference range provides some interesting uses. Using the values shown in Table 1, Listing 3 finds and returns the

sum of Bob's quiz scores. Table 13 contains a listing of each function used in Listing 3.

Listing 3. Return Bob's quiz scores.

```
=SUM(OFFSET (INDEX (A2:G16; MATCH ("Bob"; A2:A16; 0)); 0; 3; 1;  
2))
```

Table 13. Breakdown of Listing 3.

Function	Description
MATCH("Bob";A2:A16; 0)	Return 3 because Bob is the third entry in column A2:A16.
INDEX(A2:A16; 3)	Return A4:G4—the row containing Bob's quiz scores.
OFFSET(A4:G4; 0; 3; 1; 2)	Return the range D4:E4.
SUM(D4:E4)	Return the sum of Bob's quiz scores.

Tip

A simple range contains one contiguous rectangular region of cells. It is possible to define a multi-range that contains multiple simple ranges. If the reference consists of multiple ranges, you must enclose the reference or range name in parentheses.

If the reference argument to the INDEX function is a multi-range, then the range argument specifies which simple range to use (see Table 14).

Table 14. Using INDEX with a multi-range.

Function	Returns
=INDEX(B2:G2; 1; 2)	93
=INDEX(B5:G5; 1; 2)	65
=INDEX((B2:G2;B5:G5); 1; 2)	93
=INDEX((B2:G2;B5:G5); 1; 2; 1)	93
=INDEX((B2:G2;B5:G5); 1; 2; 2)	65

Database-specific Functions

Although every Calc function can be used for database manipulation, the functions in Table 15 are specifically designed for use as a database. The descriptions in Table 15 use the following terms interchangeably: row and record, cell and field, and database and all rows.

Table 15. Database functions in a Calc document.

Function	Description
DAVERAGE	Return the average of all fields that match the search criteria.
DCOUNT	Count the number of records containing numeric data that match the search criteria.

Function	Description
DCOUNTA	Count the number of records containing text data that match the search criteria.
DGET	Return the contents of a field that matches the search criteria.
DMAX	Return the maximum content of the fields that match the search criteria.
DMIN	Return the minimum content of the fields that match the search criteria.
DPRODUCT	Return the product of the fields that match the search criteria.
DSTDEV	Calculate the standard deviation using the fields that match the search criteria. The fields are treated as a sample.
DSTDEVP	Calculate the standard deviation using the fields that match the search criteria. The fields are treated as the entire population.
DSUM	Return the sum of all fields that match the search criteria.
DVAR	Calculate the variance using the fields that match the search criteria. The fields are treated as a sample.
DVARP	Calculate the variance using the fields that match the search criteria. The fields are treated as the entire population.

The syntax for the database functions is identical.

```
DCOUNT(database; database field; search criteria)
```

The database argument is the cell range that defines the database. The cell range should contain the column labels (see Listing 4). The following examples assume that the data from Table 1 is placed in Sheet 1 and the filter criteria in Table 2 is placed in Sheet 2.

Listing 4. The database argument includes the headers.

```
=DCOUNT(A1:G16; "Test 2"; Sheet2.A1:G3)
```

The database field specifies the column on which the function operates after the search criteria is applied and the data rows are selected. The database field can be specified using the column header name or as an integer. If the column is specified as an integer, 0 specifies the entire data range, 1 specifies the first column, 2 specifies the second column, and so on. Listing 5 calculates the average test score for the rows that match the search criteria.

Listing 5. "Test 2" is column 3.

```
=DAVERAGE(A1:G16; "Test 2"; Sheet2.A1:G3)
=DAVERAGE(A1:G16; 3; Sheet2.A1:G3)
```

The search criteria is the cell range containing search criteria. The search criteria is identical to the advanced filters; criteria in the same row are connected by AND and criteria in different rows are connected by OR.

Macros for a Calc Database

The following sections provide macro code examples for automating the manipulation of named ranges, database ranges, and filtering. The code has many comments and will be very helpful for those already familiar with OpenOffice macros. These examples do not include explanations of the OpenOffice Basic language and the Application Programming Interface, which are fundamental to writing macros.

Macros for Named Ranges

In a macro, a named range is accessed, created, and deleted using the `NamedRanges` property of a Calc document. Use the methods `hasByName(name)` and `getByName(name)` to verify and retrieve a named range. The method `getElementNames()` returns an array containing the names of all named ranges. The `NamedRanges` object supports the method `addNewByname`, which accepts four arguments: the name, content, position, and type. The macro in Listing 6 creates a named range, if it does not exist, which references a range of cells.

Listing 6. Create a named range that references \$Sheet1.\$B\$3:\$D\$6.

```
Sub AddNamedRange()  
    Dim oRange      ' The created range.  
    Dim oRanges     ' All named ranges.  
    Dim sName$      ' Name of the named range to create.  
    Dim oCell       ' Cell object.  
    Dim s$  
  
    sName$ = "MyNRange"  
    oRanges = ThisComponent.NamedRanges  
    If NOT oRanges.hasByName(sName$) Then  
        REM Obtain the cell address by obtaining the cell  
        REM and then extracting the address from the cell.  
        Dim oCellAddress As new com.sun.star.table.CellAddress  
        oCellAddress.Sheet = 0      'The first sheet.  
        oCellAddress.Column = 1     'Column B.  
        oCellAddress.Row = 2        'Row 3.  
  
        REM The first argument is the range name.  
        REM The second argument is the formula or expression to use.  
        REM The second argument is usually a string that  
        REM defines a range.  
        REM The third argument specifies the base address for  
        REM relative cell references.  
        REM The fourth argument is a set of flags that define  
        REM how the range is used, but most ranges use 0.  
        REM The fourth argument uses values from the  
        REM NamedRangeFlag constants (see Table 16).  
        s$ = "$Sheet1.$B$3:$D$6"  
        oRanges.addNewByname(sName$, s$, oCellAddress, 0)  
    End If  
    REM Get a range using the created named range.  
    oRange = ThisComponent.NamedRanges.getByName(sName$)
```

```

REM Print the string contained in cell $Sheet1.$B$3
oCell = oRange.getReferredCells().getCellByPosition(0,0)
Print oCell.getString()
End Sub

```

The method `addNewByname()` accepts four arguments: the name, content, position, and type. The fourth argument to the method `addNewByname()` is a combination of flags that specify how the named range will be used (see Table 16). The most common value is 0, which is not a defined constant value.

Table 16. *com.sun.star.sheet.NamedRangeFlag* constants.

Value	Name	Description
1	FILTER_CRITERIA	The range contains filter criteria.
2	PRINT_AREA	The range can be used as a print range.
4	COLUMN_HEADER	The range can be used as column headers for printing.
8	ROW_HEADER	The range can be used as row headers for printing.

The third argument, a cell address, acts as the base address for cells referenced relatively. If the cell range is not specified as an absolute address, the referenced range will differ based on where in the spreadsheet the range is used. The relative behavior is illustrated in Listing 7, which also illustrates another usage of a named range—defining an equation. The macro in Listing 7 creates the named range **AddLeft**, which refers to the equation `A3+B3` with `C3` as the reference cell. The cells `A3` and `B3` are the two cells directly to the left of `C3` so the equation `=AddLeft()` calculates the sum of the two cells directly to the left of the cell that contains the equation. Changing the reference cell to `C4`, which is below `A3` and `B3`, causes the `AddLeft` equation to calculate the sum of the two cells to the left on the previous row.

Listing 7. Create the `AddLeft` named range.

```

Sub AddNamedFunction()
    Dim oSheet          'Sheet that contains the named range.
    Dim oCellAddress    'Address for relative references.
    Dim oRanges         'The NamedRanges property.
    Dim oRange          'Single cell range.
    Dim sName As String 'Name of the equation to create.

    sName = "AddLeft"
    oRanges = ThisComponent.NamedRanges
    If NOT oRanges.hasByName(sName) Then
        oSheet = ThisComponent.getSheets().getByIndex(0)
        oRange = oSheet.getCellRangeByName("C3")
        oCellAddress = oRange.getCellAddress()
        oRanges.addNewByName(sName, "A3+B3", oCellAddress, 0)
    End If
End Sub

```

Tip

Listing 7 illustrates two capabilities that are not widely known. First, a named range can define a function. Also, the third argument acts as the base address for cells referenced relatively.

Naming Multiple Ranges

The macro in Listing 8 creates three named ranges based on the top row of a named range.

Listing 8. Create many named ranges.

```
Sub AddManyNamedRanges ()
    Dim oSheet      'Sheet that contains the named range.
    Dim oAddress    'Range address.
    Dim oRanges     'The NamedRanges property.
    Dim oRange      'Single cell range.

    oRanges = ThisComponent.NamedRanges
    oSheet = ThisComponent.getSheets().getByIndex(0)
    oRange = oSheet.getCellRangeByName("A1:C20")
    oAddress = oRange.getRangeAddress()
    oRanges.addNewFromTitles(oAddress, _
                            com.sun.star.sheet.Border.TOP)
End Sub
```

The constants in Table 17 determine the location of the headers when multiple ranges are created using the method `addNewFromTitles()`.

Table 17. com.sun.star.sheet.Border constants.

Value	Name	Description
0	TOP	Select the top border.
1	BOTTOM	Select the bottom border.
2	RIGHT	Select the right border.
3	LEFT	Select the left border.

Caution

It is possible to create multiple named ranges with the same name. Creating multiple ranges with a single command increases the likelihood that multiple ranges will be created with the same name. Avoid this if possible.

Macro for a Database Range

In a macro, a database range is accessed, created, and deleted from the `DatabaseRanges` property. The macro in Listing 9 creates a database range named `MyName` and sets the range to be used as an auto filter.

Listing 9. Create a database range and an auto filter.

```
Sub AddNewDatabaseRange ()
```

```

Dim oRange 'DatabaseRange object.
Dim oAddr 'Cell address range for the database range.
Dim oSheet 'First sheet, which will contain the range.
Dim oDoc 'Reference ThisComponent with a shorter name.

oDoc = ThisComponent
If NOT oDoc.DatabaseRanges.hasByName("MyName") Then
    oSheet = ThisComponent.getSheets().getByIndex(0)
    oRange = oSheet.getCellRangeByName("A1:F10")
    oAddr = oRange.getRangeAddress()
    oDoc.DatabaseRanges.addNewByName("MyName", oAddr)
End If
oRange = oDoc.DatabaseRanges.getByName("MyName")
oRange.AutoFilter = True
End Sub

```

Macros for Filtering

The macro in Listing 10 creates a simple filter for the first sheet.

Listing 10. Create a simple sheet filter.

```

Sub SimpleSheetFilter()
    Dim oSheet ' Sheet that will contain the filter.
    Dim oFilterDesc ' Filter descriptor.
    Dim oFields(0) As New com.sun.star.sheet.TableFilterField

    oSheet = ThisComponent.getSheets().getByIndex(0)

    REM If argument is True, creates an empty filter
    REM descriptor. If argument is False, create a
    REM descriptor with the previous settings.
    oFilterDesc = oSheet.createFilterDescriptor(True)

    With oFields(0)
        REM You could use the Connection property to indicate
        REM how to connect to the previous field. This is
        REM the first field, so this is not required.
        '.Connection = com.sun.star.sheet.FilterConnection.AND
        '.Connection = com.sun.star.sheet.FilterConnection.OR

        REM The Field property is the zero based column
        REM number to filter. If you have the cell, you
        REM can use .Field = oCell.CellAddress.Column.
        .Field = 5

        REM Compare using a numeric or a string?
        .IsNumeric = True

        REM The NumericValue property is used
        REM because .IsNumeric = True from above.
        .NumericValue = 80
    End With
End Sub

```

```

    REM If IsNumeric was False, then the
    REM StringValue property would be used.
    REM .StringValue = "what ever"

    REM Valid operators include EMPTY, NOT_EMPTY, EQUAL,
    REM NOT_EQUAL, GREATER, GREATER_EQUAL, LESS,
    REM LESS_EQUAL, TOP_VALUES, TOP_PERCENT,
    REM BOTTOM_VALUES, and BOTTOM_PERCENT
    .Operator = com.sun.star.sheet.FilterOperator.GREATER_EQUAL
End With

REM The filter descriptor supports the following
REM properties: IsCaseSensitive, SkipDuplicates,
REM UseRegularExpressions,
REM SaveOutputPosition, Orientation, ContainsHeader,
REM CopyOutputData, OutputPosition, and MaxFieldCount.
oFilterDesc.setFilterFields(oFields())
oFilterDesc.ContainsHeader = True
oSheet.filter(oFilterDesc)
End Sub

```

When a filter is applied to a sheet, it replaces any existing filter for that sheet. Setting an empty filter in a sheet will therefore remove all filters for the sheet (see Listing 11).

Listing 11. Remove the current sheet filter.

```

Sub RemoveSheetFilter()
    Dim oSheet          ' Sheet to filter.
    Dim oFilterDesc      ' Filter descriptor.

    oSheet = ThisComponent.getSheets().getByIndex(0)
    oFilterDesc = oSheet.createFilterDescriptor(True)
    oSheet.filter(oFilterDesc)
End Sub

```

Listing 12 demonstrates a more advanced filter that filters two columns and uses regular expressions. Some behavior of Listing 12 may be unexpected. Although you can create a filter descriptor using any sheet cell range, the filter applies to the entire sheet. This is seen in the code

```
.Field = 0 ' Filter column A
```

where column A is referred to with the index 0, that is, as the first column, though the Filter Descriptor was constructed from the cell range E12:G19, and you might expect column E to have the index of zero.

Listing 12. A simple sheet filter using two columns.

```

Sub SimpleSheetFilter_2()
    Dim oSheet          ' Sheet to filter.
    Dim oRange          ' Range to be filtered.
    Dim oFilterDesc      ' Filter descriptor.
    Dim oFields(1) As New com.sun.star.sheet.TableFilterField

    oSheet = ThisComponent.getSheets().getByIndex(0)
    oRange = oSheet.getCellRangeByName("E12:G19")

```

```

REM If argument is True, creates an
REM empty filter descriptor.
oFilterDesc = oRange.createFilterDescriptor(True)

REM Setup a field to view cells with content that
REM start with the letter b.
With oFields(0)
    .Field = 0          ' Filter column A.
    .IsNumeric = False  ' Use a string, not a number.
    .StringValue = "b.*" ' Everything starting with b.
    .Operator = com.sun.star.sheet.FilterOperator.EQUAL
End With
REM Setup a field that requires both conditions and
REM this new condition requires a value greater or
REM equal to 70.
With oFields(1)
    .Connection = com.sun.star.sheet.FilterConnection.AND
    .Field = 5          ' Filter column F.
    .IsNumeric = True   ' Use a number
    .NumericValue = 70   ' Values greater than or equal to 70
    .Operator = com.sun.star.sheet.FilterOperator.GREATER_EQUAL
End With

oFilterDesc.setFilterFields(oFields())
oFilterDesc.ContainsHeader = False
oFilterDesc.UseRegularExpressions = True
oSheet.filter(oFilterDesc)
End Sub

```

Macros for Advanced Filters

Defining an advanced filter with a macro requires only getting the cell ranges for the cells to be filtered, the cell range of the filter criteria, and using the method `createFilterDescriptorByObject()`. In Listing 13, they are in A1:G13 of the first sheet, and the filter criteria are in A1:G3 of the second sheet.

Listing 13. Use an advanced filter.

```

Sub UseAnAdvancedFilter()
    Dim oSheet      'A sheet from the Calc document.
    Dim oRanges      'The NamedRanges property.
    Dim oCritRange   'Range that contains the filter criteria.
    Dim oDataRange   'Range that contains the data to filter.
    Dim oFiltDesc    'Filter descriptor.

    REM Range that contains the filter criteria
    oSheet = ThisComponent.getSheets().getByIndex(1)
    oCritRange = oSheet.getCellRangeByName("A1:G3")

    REM You can also obtain the range containing the
    REM filter criteria from a named range.
    REM oRanges = ThisComponent.NamedRanges

```

```

REM oRange = oRanges.getByName("AverageLess80")
REM oCritRange = oRange.getReferredCells()

REM The data that you want to filter
oSheet = ThisComponent.getSheets().getByIndex(0)
oDataRange = oSheet.getCellRangeByName("A1:G16")

oFiltDesc = oCritRange.createFilterDescriptorByObject(oDataRange)
oDataRange.filter(oFiltDesc)
End Sub

```

Change properties on the filter descriptor to change the filter's behavior (see Table 18).

The filter created in Listing 13 filters the data in place. Modify the `OutputPosition` property to specify a different output position (see Listing 14). The filter descriptor must be modified before the filter is applied.

Table 18. Advanced filter properties.

Property	Comment
ContainsHeader	Boolean (true or false) component that specifies if the first row (or column) contains headers that should not be filtered.
CopyOutputData	Boolean that specifies if the filtered data should be copied to another position in the document.
IsCaseSensitive	Boolean which specifies if the case of letters is important when comparing entries.
Orientation	Specifies if columns (<code>com.sun.star.table.TableOrientation.COLUMNS</code>) or rows (<code>com.sun.star.table.TableOrientation.ROWS</code>) are filtered.
OutputPosition	If <code>CopyOutputData</code> is True specifies the position where filtered data is to be copied.
SaveOutputPosition	Boolean that specifies if the <code>OutputPosition</code> position is saved for future calls.
SkipDuplicates	Boolean that specifies if duplicate entries are left out of the result.
UseRegularExpressions	Boolean that specifies if the filter strings are interpreted as regular expressions.

Listing 14. Copy filtered results to a different location.

```

REM Copy the output data rather than filter in place.
oFiltDesc.CopyOutputData = True

REM Create a CellAddress and set it for Sheet3,
REM Column B, Row 4 (remember, start counting with 0)
Dim x As New com.sun.star.table.CellAddress

```



```
x.Sheet = 2
x.Column = 1
x.Row = 3
oFiltDesc.OutputPosition = x
```

Tip

A struct is a data type in many computer languages; it is analogous to a [record](#). A struct can have one or more members; each of these may be of different data types. Structs are used to group associated data together.

(Advanced material.) The `OutputPosition` property returns a copy of a struct. Because a copy is returned, setting the individual values directly is not possible. For example, `oFiltDesc.OutputPosition.Row = 2` does not work (because you set the `Row` on the copy to 2, but do not change the original). To change the `OutputPosition`, you need to make a new struct and assign that to the `FilterDescriptor`, as was done in Listing 14.